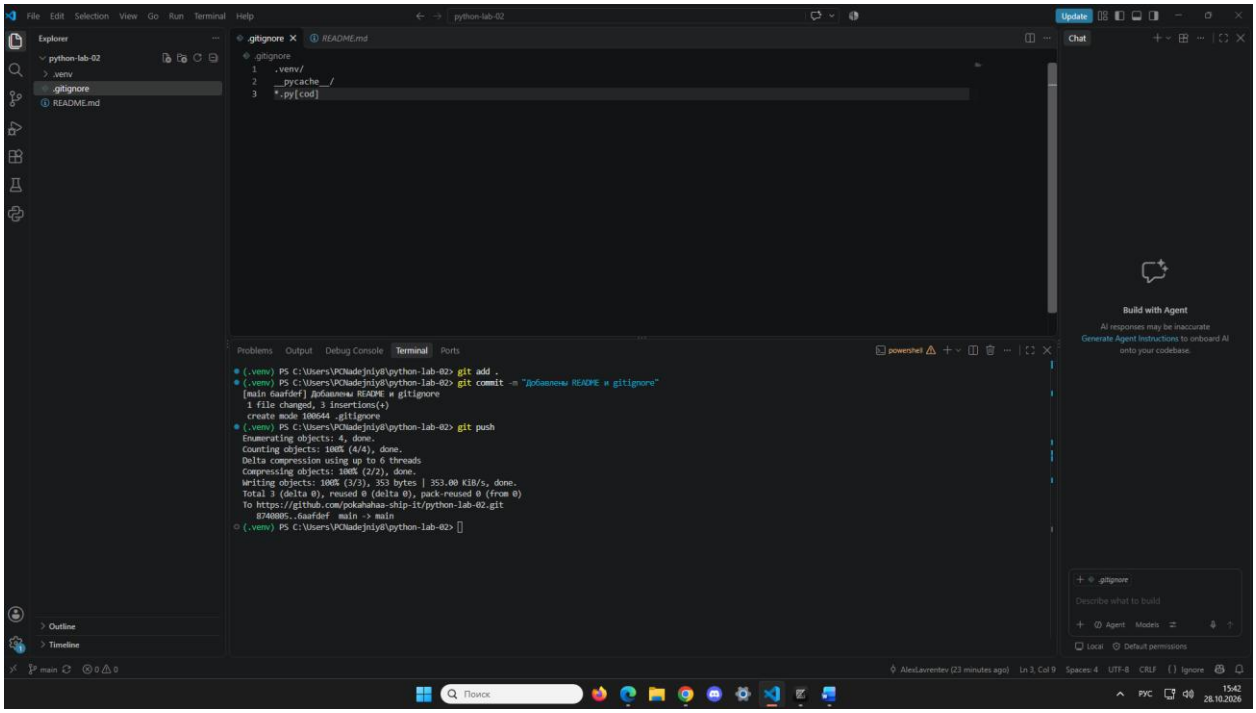
https://github.com/pokahahaa-ship-it/python-lab-02.git

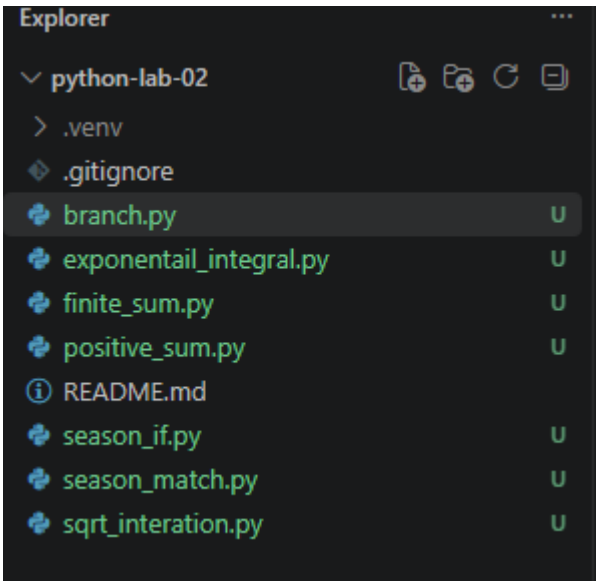
Лабораторная 2. Ветвления и циклы в Python

Цель работы: освоить выбор действий с помощью if-elif-else и простейшего math-case, перебор с помощью for и range(), циклы while, операторы break и continue; научиться проверять границы условий и завершение циклов.

Создал новую папку(python-lab-02) в Visual studio code в ней создал файл .gitignore. Также видно, что я сделал коммит и отправил на GitHub без ошибок, чтоб в дальнейшем не возникало проблем



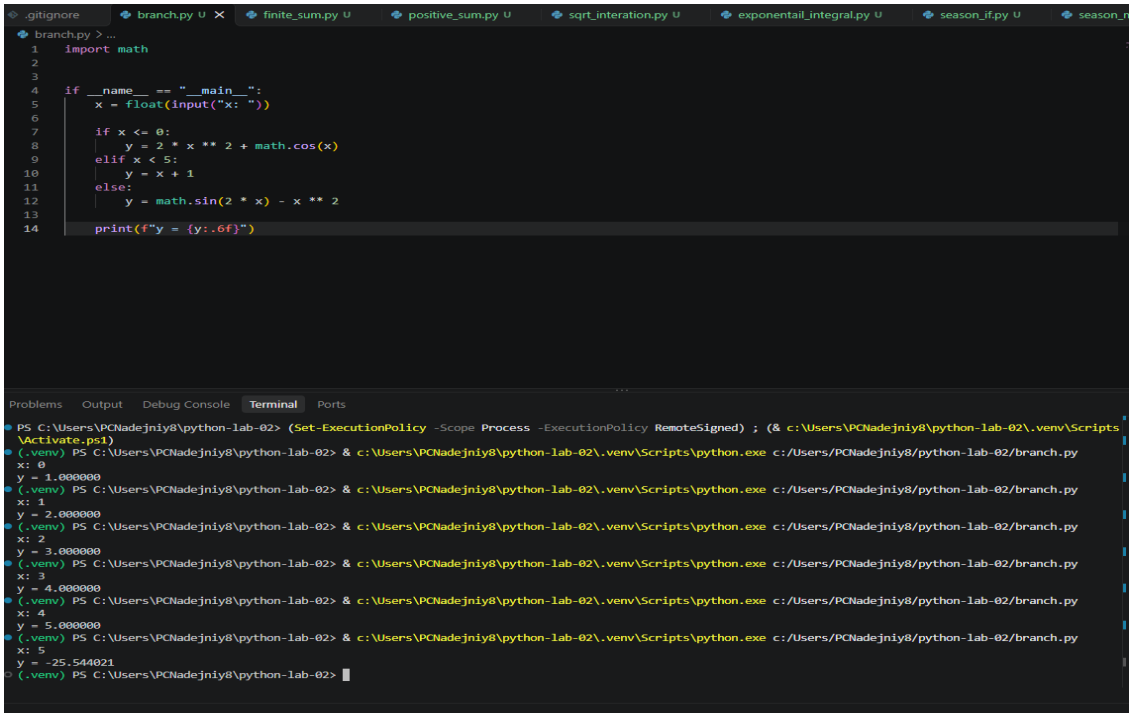




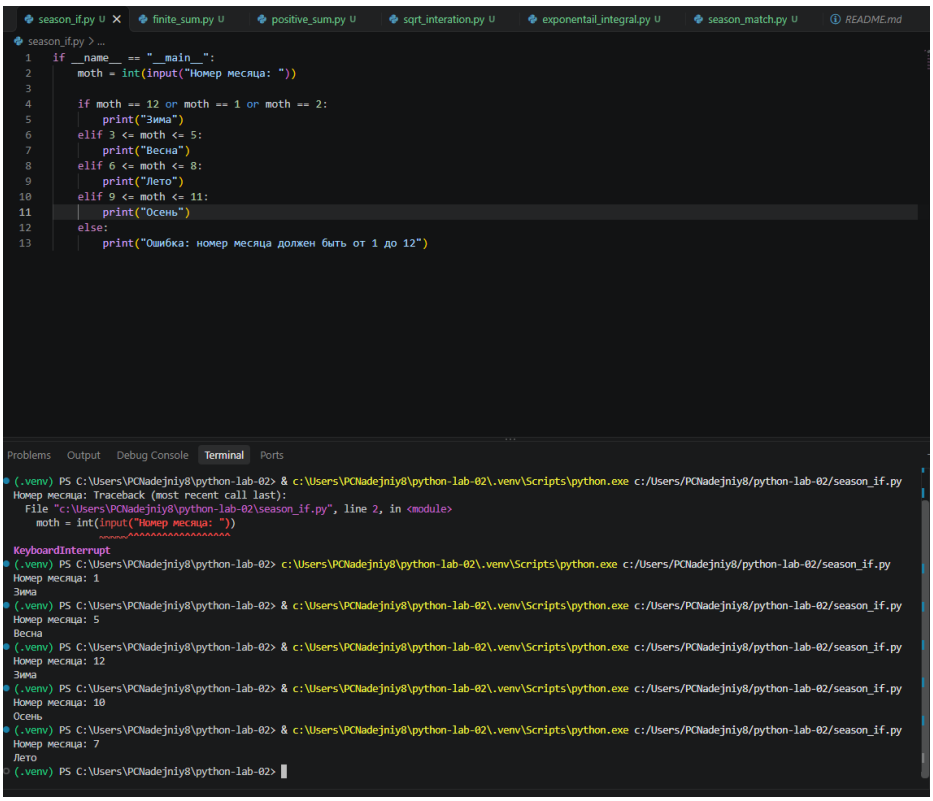
Пример 1. Кусочная функция – btanch.py

Была дана функция, написал код и проверил значения от 0 до 5.

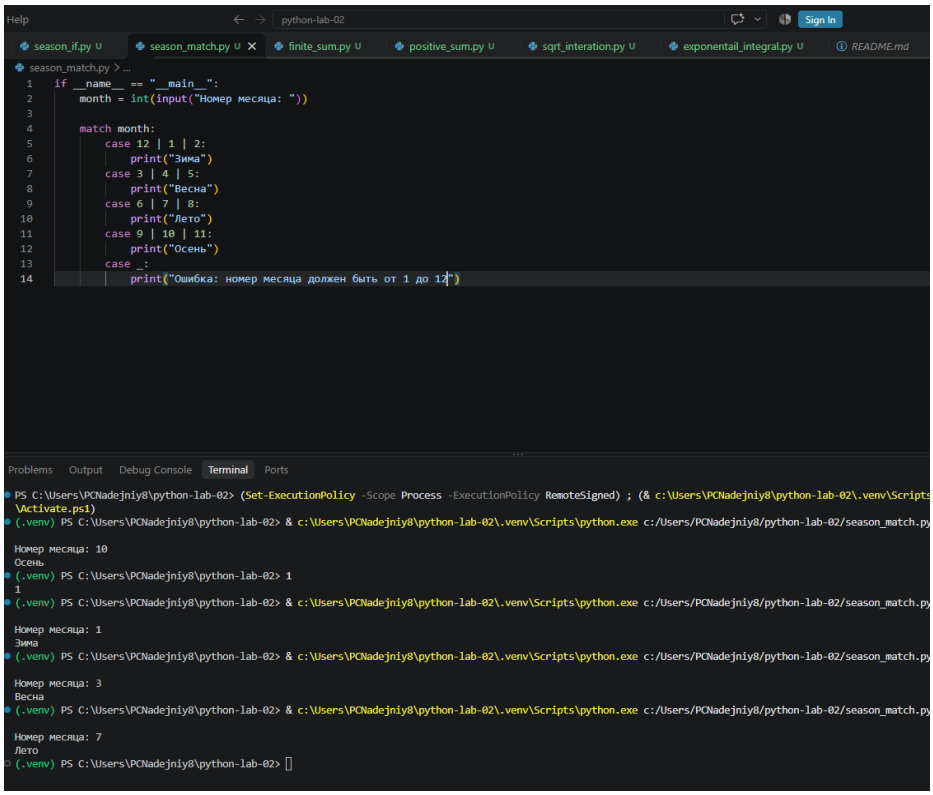
$$y = \begin{cases} 2x^2 + \cos x, & x \le 0, \\ x + 1, & 0 < x < 5, \\ \sin(2x) - x^2, & x \ge 5. \end{cases}$$



Пример 2. Выбор сезона через if – season_if.py

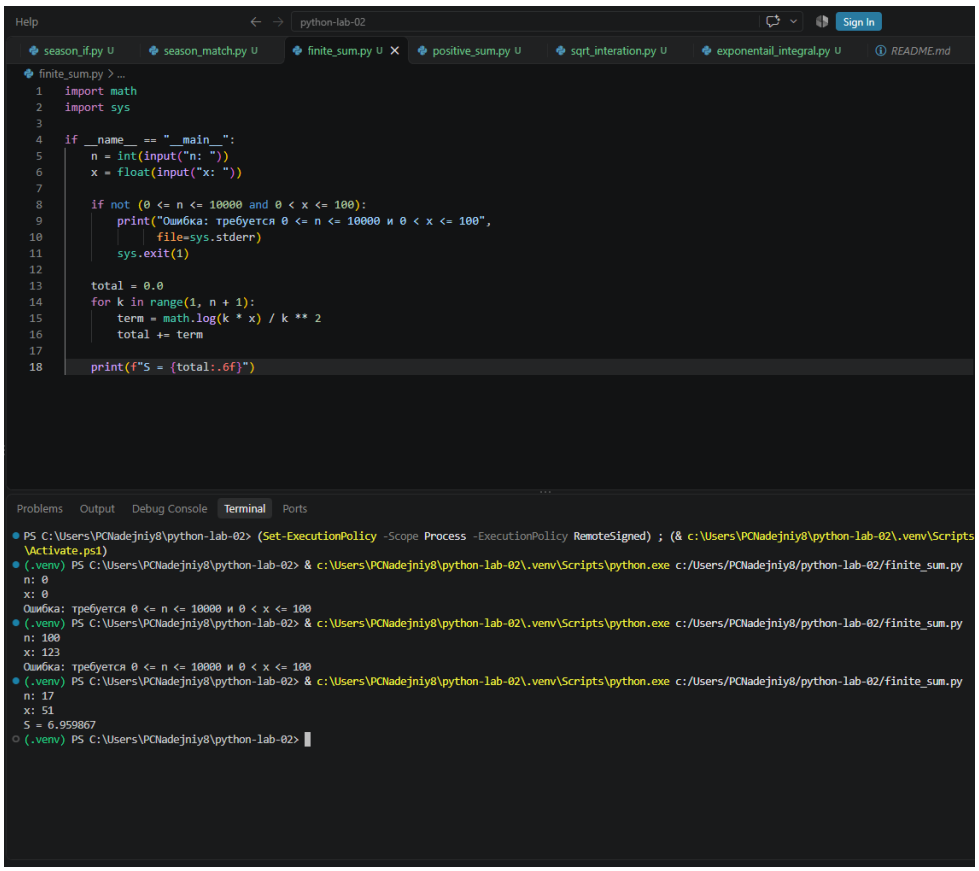


Пример 3. Та же задача через match – season_match.py

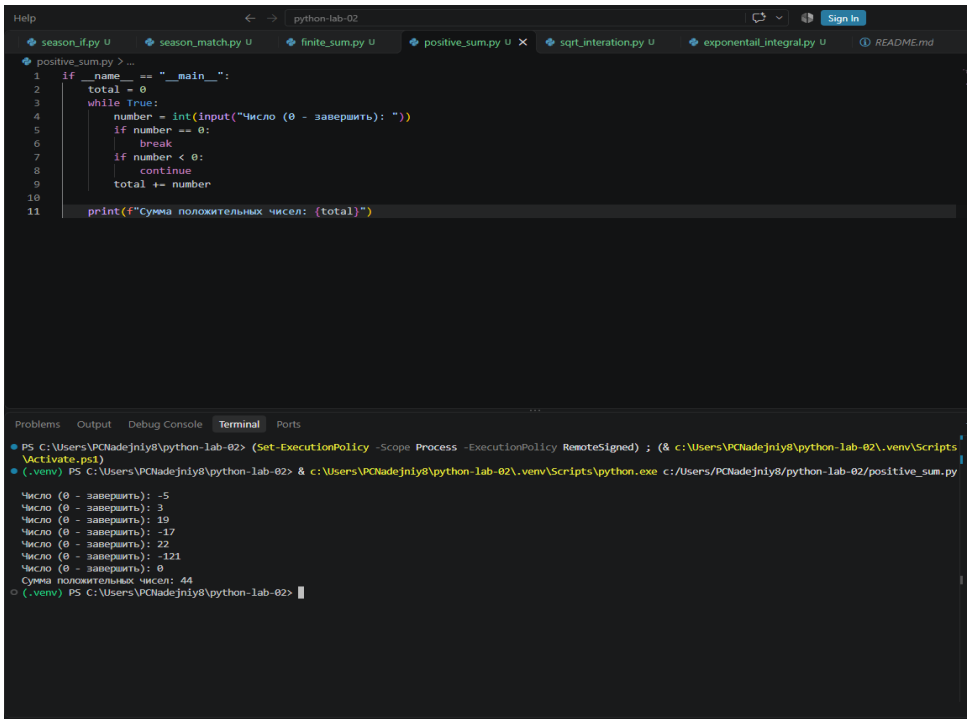


Пример 4. Конечная сумма – finite_sum.py

$$S = \sum_{k=1}^{n} \frac{\ln(kx)}{k^2}, \qquad 0 \le n \le 10000, \quad 0 < x \le 100.$$

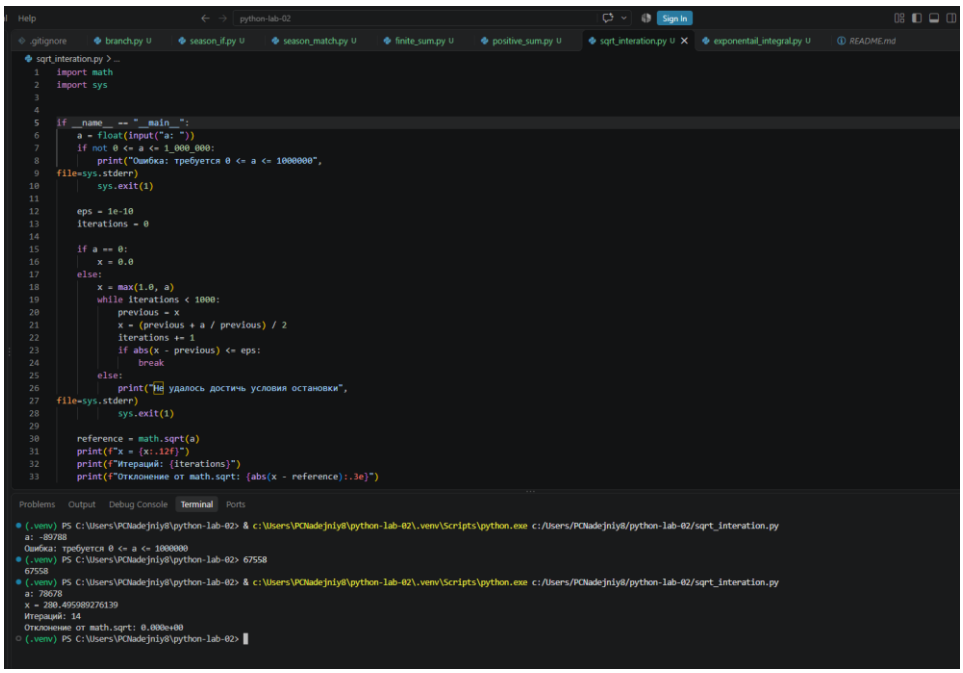


Пример 5. Ввод по признака завершения – positive_sum.py



Пример 6. Итерационное вычисление корня – sqrt_iteration.py

$$x_{k+1} = \frac{x_k + a/x_k}{2}, \qquad x_0 = \max(1, a).$$



```python
import math
import sys


if __name__ == "__main__":
    a = float(input("a: "))
    if not 0 <= a <= 1_000_000:
        print("Ошибка: требуется 0 <= a <= 1000000",
file=sys.stderr)
        sys.exit(1)

    eps = 1e-10
    iterations = 0

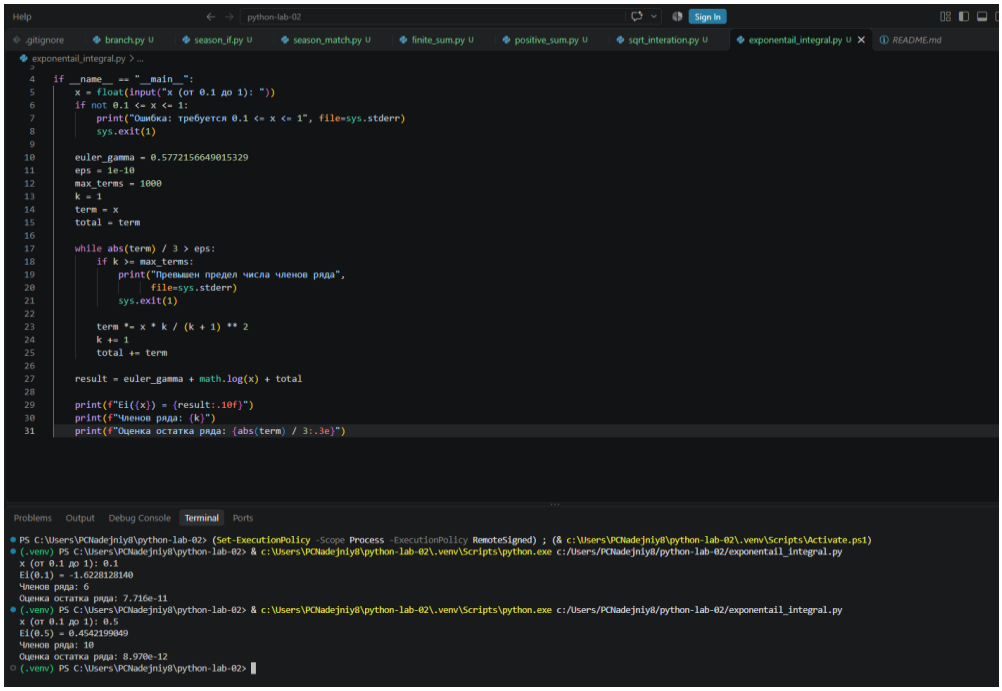
    if a == 0:
        x = 0.0
    else:
        x = max(1.0, a)
        while iterations < 1000:
            previous = x
            x = (previous + a / previous) / 2
            iterations += 1
            if abs(x - previous) <= eps:
                break
        else:
            print("Не удалось достичь условия остановки",
file=sys.stderr)
            sys.exit(1)

    reference = math.sqrt(a)
    print(f"x = {x:.12f}")
    print(f"Итераций: {iterations}")
    print(f"Отклонение от math.sqrt: {abs(x - reference):.3e}")
```

```
(.venv) PS C:\Users\PCNadejniy8\python-lab-02> & c:\Users\PCNadejniy8\python-lab-02\.venv\Scripts\python.exe c:/Users/PCNadejniy8/python-lab-02/sqrt_interation.py
a: -89788
Ошибка: требуется 0 <= a <= 1000000
(.venv) PS C:\Users\PCNadejniy8\python-lab-02> 67558
67558
(.venv) PS C:\Users\PCNadejniy8\python-lab-02> & c:\Users\PCNadejniy8\python-lab-02\.venv\Scripts\python.exe c:/Users/PCNadejniy8/python-lab-02/sqrt_interation.py
a: 78678
x = 280.495989276139
Итераций: 14
Отклонение от math.sqrt: 0.000e+00
(.venv) PS C:\Users\PCNadejniy8\python-lab-02>
```

Пример 7. Интегральная показательная функция через ряд – exponential_integral.py

$$\mathrm{Ei}(x) = \gamma + \ln x + \int_0^x \frac{e^t - 1}{t}\,dt = \gamma + \ln x + \sum_{k=1}^{\infty} \frac{x^k}{k \cdot k!},$$



```python
if __name__ == "__main__":
    x = float(input("x (от 0.1 до 1): "))
    if not 0.1 <= x <= 1:
        print("Ошибка: требуется 0.1 <= x <= 1", file=sys.stderr)
        sys.exit(1)

    euler_gamma = 0.5772156649015329
    eps = 1e-10
    max_terms = 1000
    k = 1
    term = x
    total = term

    while abs(term) / 3 > eps:
        if k >= max_terms:
            print("Превышен предел числа членов ряда",
                  file=sys.stderr)
            sys.exit(1)

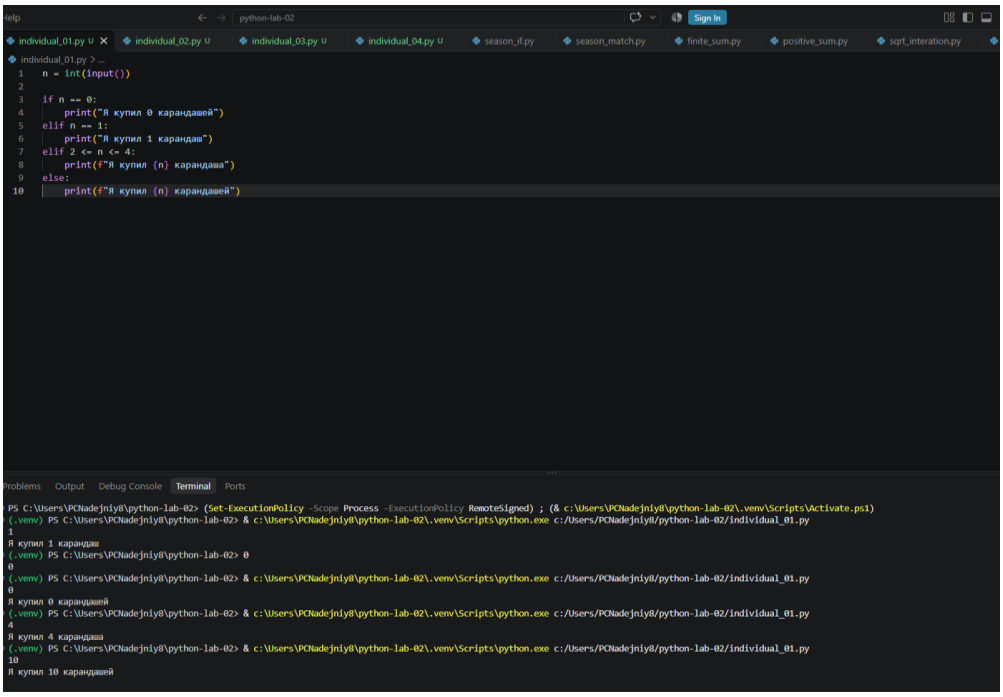
        term *= x * k / (k + 1) ** 2
        k += 1
        total += term

    result = euler_gamma + math.log(x) + total

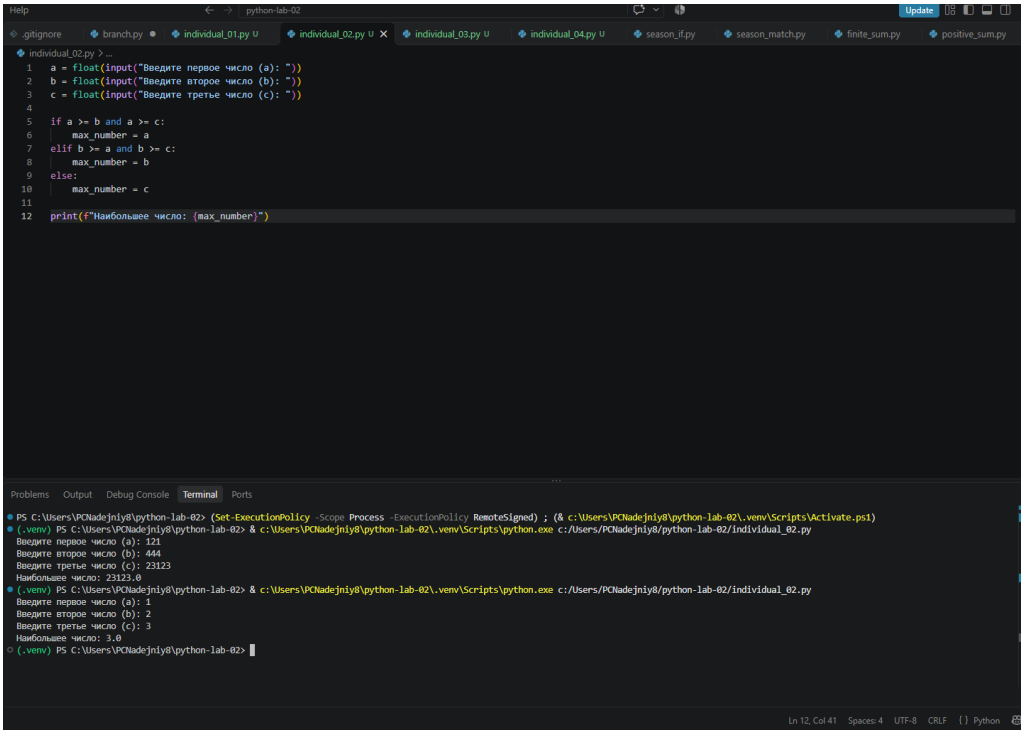
    print(f"Ei({x}) = {result:.10f}")
    print(f"Членов ряда: {k}")
    print(f"Оценка остатка ряда: {abs(term) / 3:.3e}")
```

```
PS C:\Users\PCNadejniy8\python-lab-02> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& c:\Users\PCNadejniy8\python-lab-02\.venv\Scripts\Activate.ps1)
(.venv) PS C:\Users\PCNadejniy8\python-lab-02> & c:\Users\PCNadejniy8\python-lab-02\.venv\Scripts\python.exe c:/Users/PCNadejniy8/python-lab-02/exponentail_integral.py
x (от 0.1 до 1): 0.1
Ei(0.1) = -1.6228128140
Членов ряда: 6
Оценка остатка ряда: 7.716e-11
(.venv) PS C:\Users\PCNadejniy8\python-lab-02> & c:\Users\PCNadejniy8\python-lab-02\.venv\Scripts\python.exe c:/Users/PCNadejniy8/python-lab-02/exponentail_integral.py
x (от 0.1 до 1): 0.5
Ei(0.5) = 0.4542199049
Членов ряда: 10
Оценка остатка ряда: 8.970e-12
(.venv) PS C:\Users\PCNadejniy8\python-lab-02>
```

Индивидуальное задание 1. Вариант 10, нужно было использовать if-elif-else. Вывести фразу Я купил N карандашей

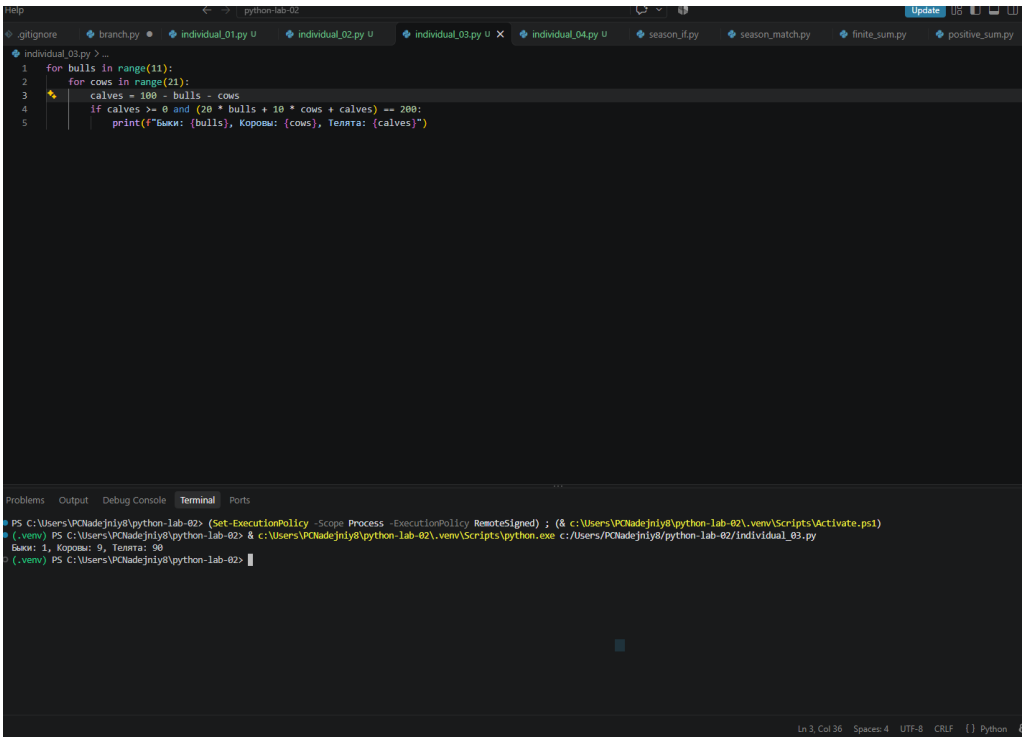


Индивидуальное задание 2. Вариант 10, нужно было использовать if-elif-else. Вывести на экран большее из трёх заданных чисел. Не используя max().

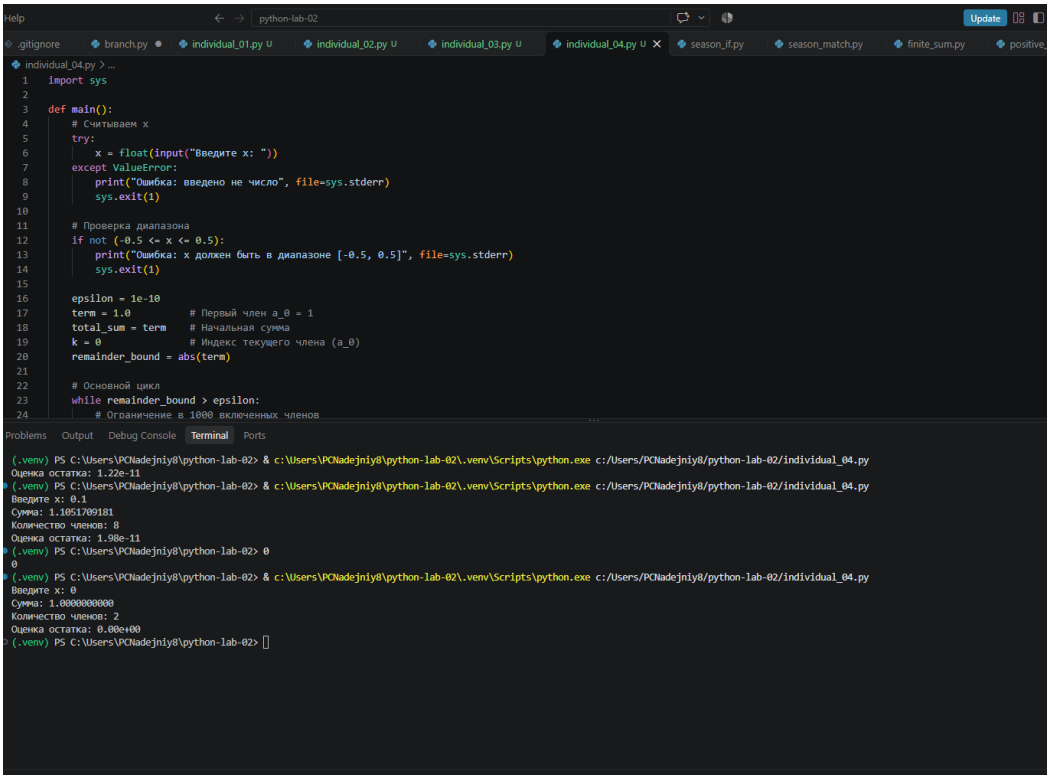


Индивидуальное задание 3. Вариант 10, использовать for или while.

Условие: сколько можно купить быков, коров и телят, платя за быка 10р, за корову 5р, а за теленка 0,5р, если на 100р надо купить 100 голов скота.



Индивидуальное задание 4. Вариант 10. Показательная функция, допустимый диапазон: -0,5 <= x <= 0,5



Вывод: освоил выбор действий с помощью if-elif-else и простейшего math-case, перебор с помощью for и range(), циклы while, операторы break и continue; научился проверять границы условий и завершение циклов.

Контрольные вопросы:

Основы синтаксиса и логики

1. Чем различаются присваивание и сравнение? Присваивание (=) записывает значение в переменную (например, x = 5). Сравнение (==) проверяет, равны ли два значения, и возвращает логический результат True или False (например, x == 5).

2. Какие значения считаются ложными? Почему строка "0" истинна? Ложными (False) считаются: число 0 (и 0.0), пустые коллекции (пустая строка "", пустой список [], пустой словарь {}), а также None и False. Строка "0" истинна, потому что она не пустая. В ней есть один символ. Python смотрит на длину строки, а не на то, что внутри написана цифра ноль.

3. Как работают and, or, not и сокращённое вычисление?

- and (И): возвращает True, только если оба условия истинны.
- or (ИЛИ): возвращает True, если хотя бы одно условие истинно.
- not (НЕ): инвертирует значение (меняет True на False и наоборот).
- *Сокращённое вычисление (short-circuit):* Python не вычисляет вторую часть выражения, если результат уже понятен по первой. Например, в A and B, если A ложно, то B даже не будет проверяться.

4. Чем несколько if отличаются от цепочки if-elif-else? Несколько независимых if проверяются все подряд. Если несколько условий истинны, выполнятся все блоки. В цепочке if-elif-else проверка идет сверху вниз, и как только находится первое истинное условие, выполняется его блок, а все остальные пропускаются.

5. Почему порядок условий важен при пересекающихся диапазонах? Потому что в if-elif срабатывает только первое подходящее условие. Если вы сначала проверите x > 0, а потом x > 10, то для числа 15 сработает первое условие, и программа не дойдет до второго, даже если оно более точное.

6. Что такое условное выражение и когда оно удобно? Это тернарный оператор: A if условие else B. Он возвращает значение A, если условие истинно, и B, если ложно. Удобно использовать для коротких присваиваний в одну строку (например: status = "Взрослый" if age >= 18 else "Ребенок").

7. С какой версии Python доступен match? С версии Python 3.10.

Конструкция match-case (сопоставление с образцом)

8. Чем числовой образец 1 отличается от строкового "1"? Типом данных. 1 — это целое число (int), а "1" — это строка (str). В match они не будут считаться совпадением.

9. Что означает case _ и почему он размещается последним? case _ — это шаблон по умолчанию (аналог else), он совпадает с чем угодно. Размещается последним, потому что если поставить его выше, он «перехватит» управление и остальные ветви кода станут недостижимыми.

10. Как объединить несколько литералов в одной ветви case? Использовать оператор вертикальной черты | (или). Например: case 1 | 2 | 3:.

11. Нужен ли break после case? Что произойдёт при отсутствии совпадений?
Нет, break не нужен (в отличие от switch в C/Java), Python не «проваливается» в следующий case. Если ни одно совпадение не найдено и нет case _, то просто ничего не происходит, программа идет дальше.

12. Почему case name не сравнивает значение с переменной name? В match-case имя без точки считается «захватом» (capture pattern). То есть case name: просто присвоит в переменную name проверяемое значение. Чтобы сравнить с переменной, нужно использовать точечную запись (например, case Constants.name:).

13. Когда удобнее использовать if, а когда простейший match? if удобнее для математических сравнений (x > 5, a < b) и сложной логики. match удобнее, когда нужно проверить одну переменную на совпадение с набором конкретных значений, типов или структур.

Циклы (for, while)

14. Как работает for и что возвращает range? for перебирает элементы итерируемого объекта по очереди. range() возвращает не список, а специальный ленивый объект (генератор), который выдает числа по одному по мере необходимости, экономя память.

15. Как получить 15, 13, ..., 1? Почему ноль в этой последовательности отсутствует?
Код: range(15, 0, -2).
Ноль отсутствует, потому что шаг равен -2. После 1 следующим числом было бы -1 (1 - 2 = -1), но мы указали границу 0, до которой (не включая её) нужно идти. Так как 1 минус 2 не равно 0, ноль выпадает.

16. Когда while не выполнит ни одной итерации?
Если его условие изначально ложно (например, while 5 > 10: или while False:).

17. Чем отличаются break и continue? На какой вложенный цикл они действуют?
break полностью прерывает цикл. continue пропускает текущую итерацию и переходит к следующей. Они действуют только на самый внутренний (текущий) цикл, в котором они находятся.

18. Как continue может привести к бесконечному циклу?
Если continue стоит *до* изменения переменной-счетчика в while. Например, если while i < 5: и внутри if x: continue, а строчка i += 1 стоит после continue, то i никогда не увеличится, и условие всегда будет истинным.

19. Когда выполняется else у цикла?
Блок else у циклов for или while выполняется тогда, когда цикл завершился естественным путем (условие стало ложным или элементы кончились). Если цикл был прерван через break, блок else не выполнится.

Алгоритмы и вычисления

20. Как выбирается начальное значение накопителя суммы и произведения?
Для суммы накопитель начинается с 0 (потому что 0 + любое число = это число). Для произведения — с 1 (потому что 1 * любое число = это число). Если начать произведение с 0, результат всегда будет 0.

21. Чем условие остановки численного алгоритма отличается от оценки погрешности?
Условие остановки — это программное правило (например, abs(term) < eps), по которому мы прекращаем цикл. Оценка погрешности (remainder_bound) — это математическая величина, показывающая, насколько наш ответ может отличаться от истинного. Условие остановки опирается на оценку.

22. Почему приближённые вещественные результаты иногда сравнивают через math.isclose()?
Из-за ошибок округления чисел с плавающей точкой (float). В Python 0.1 + 0.2 не равно ровно 0.3. Поэтому сравнивать их через == опасно. math.isclose() проверяет, находятся ли числа в пределах заданной крошечной погрешности друг от друга.

Тестирование и отладка

23. Как составить тесты для проверки границ ветвления? Использовать метод граничных значений. Если условие x > 10, нужно проверить три числа: 9 (до границы), 10 (сама граница) и 11 (после границы).

24. Что показывает трассировочная таблица и чем она дополняет конечный вывод?
Она показывает пошаговое изменение значений всех переменных на каждой итерации цикла. Конечный вывод показывает только результат, а таблица позволяет увидеть, *как* программа к нему пришла, и найти логическую ошибку.

25. Чем отличаются stdout, stderr и код завершения? Почему сообщение об ошибке не останавливает программу автоматически?

- stdout — стандартный поток для обычного вывода (туда идет print).
- stderr — поток для сообщений об ошибках.
- Код завершения — число, которое программа возвращает ОС (0 — успех, 1 и выше — ошибка).
  Просто вывод текста в stderr — это лишь печать текста в другое окно консоли. Это не является исключением (Exception), поэтому программа продолжит выполняться. Чтобы остановить её, нужно явно вызвать sys.exit(1) или возбудить ошибку.